



RETRIEVAL-AUGMENTED GENERATION

Chatbot ตอบคำถามเชิงลึกและข้อมูลเฉพาะทาง ด้วยเทคนิค RAG

จากคลังความรู้เฉพาะทาง สู่ Chatbot ที่ตอบได้ตรงประเด็น



ภาพรวมโครงสร้างการบรรยาย



01

ทำไมต้องรู้เรื่องนี้

- Motivation



02

RAG คืออะไร

- Concept



03

องค์ประกอบของ RAG

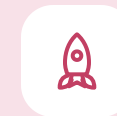
- Architecture



04

เครื่องมือ สำเร็จรูป

- Tools landscape



05

Workshop

- ลงมือสร้างจริง



ทำไมต้องรู้เรื่องนี้

- 01 ข้อจำกัดของ LLM ล้วน ๆ** — ตอบจากความรู้ที่ถูก train มาเท่านั้น ไม่รู้ข้อมูลใหม่หรือข้อมูลภายในองค์กร และมีโอกาส "hallucinate" (มโนคำตอบ) เมื่อถูกถามเรื่องที่ไม่มีในความรู้
- 02 สถานการณ์ที่ต้องการคำตอบเฉพาะทาง** — คู่มือปฏิบัติงานภายใน, ระเบียบ/ข้อบังคับเฉพาะหน่วยงาน, เอกสารวิชาการ/งานวิจัยเฉพาะสาขา, FAQ ของหลักสูตร
- 03 ทางเลือกที่มีอยู่ก่อน RAG และข้อจำกัด** — Fine-tuning (ค่าใช้จ่ายสูง ต้อง train โมเดลใหม่ทุกครั้งที่ข้อมูลเปลี่ยน) หรือยัดข้อมูลทั้งหมดใน prompt (จำกัดด้วย context window ค่าใช้จ่ายต่อ request สูงขึ้น)
- 04 RAG คือคำตอบตรงกลาง** — ใช้ LLM ที่มีอยู่แล้ว (ไม่ต้อง fine-tune) + หาข้อมูลที่เกี่ยวข้องมาป้อนให้ก่อนตอบ ทำให้ตอบได้ทันเวลา อัปเดตง่าย และอ้างอิงแหล่งที่มาได้



RAG คืออะไร

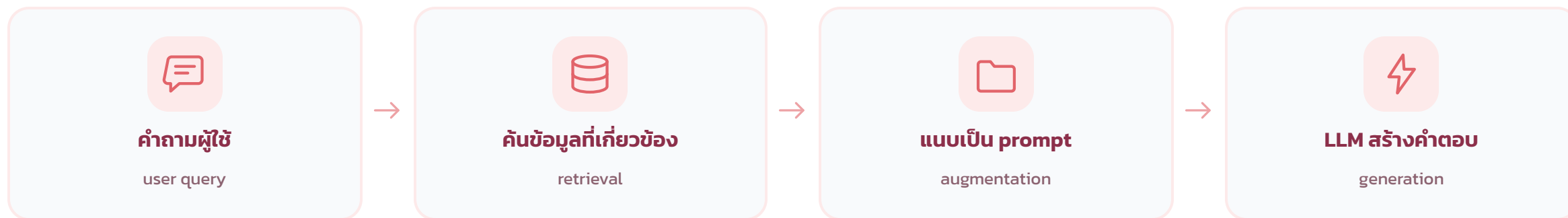
01 นิยาม — Retrieval-Augmented Generation = เทคนิคที่ผสาน "การค้นคืนข้อมูล" (Retrieval) เข้ากับ "การสร้างคำตอบด้วย LLM" (Generation)

02 เปรียบเทียบให้เห็นภาพ — เหมือนเปิดหนังสือสอบ (open-book exam) แทนที่จะให้ LLM ตอบจากความจำล้วน ๆ (closed-book)



หลักการทำงานแบบภาพรวม

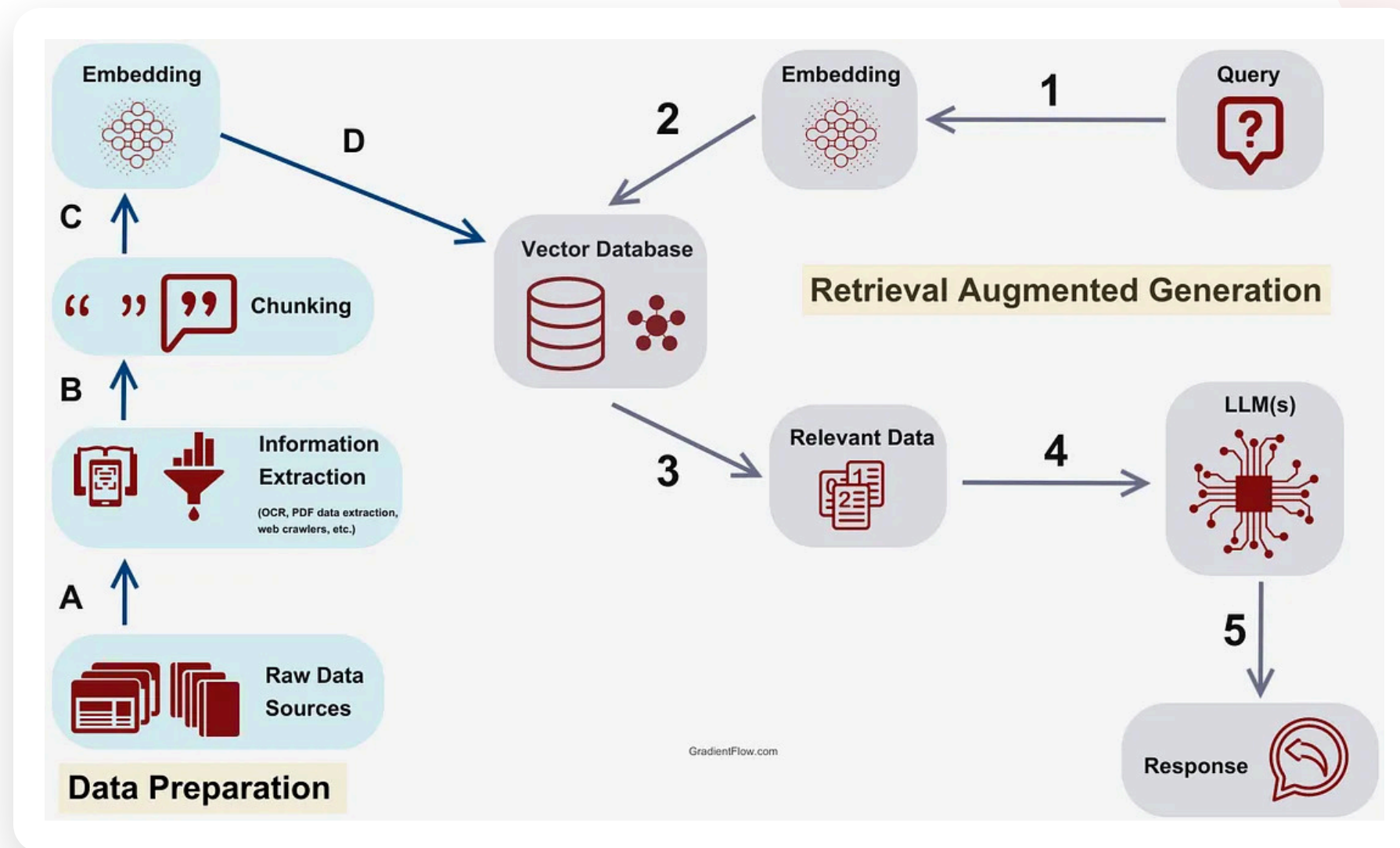
flow 4 ขั้นตอน





ข้อดีหลักของ RAG

- 01 **ลด Hallucination** — คำตอบอิงจากข้อมูลที่ค้นคืนมาได้จริง ไม่ใช่ความจำล้วน ๆ ของโมเดล
- 02 **อัปเดตความรู้ได้ง่าย** — เพิ่ม/แก้ไขเอกสารในคลังความรู้ได้ทันที โดยไม่ต้อง train โมเดลใหม่
- 03 **อ้างอิงแหล่งที่มาได้ (Traceability)** — ตรวจสอบย้อนกลับได้ว่าคำตอบมาจากเอกสารใด
- 04 **ควบคุมขอบเขตคำตอบได้** — กำหนดได้ว่าจะตอบจากคลังข้อมูลใด ในรูปแบบและภาษาใด





องค์ประกอบของ RAG

ส่วนประกอบหลักที่ต้องมี

องค์ประกอบ	หน้าที่
Document Ingestion	นำเอกสารต้นทาง (PDF, DOCX, PPTX, TXT ฯลฯ) เข้าระบบ
Chunking	ตัดแบ่งเอกสารเป็นชิ้นย่อย ๆ เพื่อให้ค้นคืนได้แม่นยำ
Embedding Model	แปลงข้อความเป็นเวกเตอร์เพื่อวัดความใกล้เคียงเชิงความหมาย
Vector Database	เก็บเวกเตอร์เพื่อค้นคืนแบบ similarity search (เช่น NanoVectorDB)
Knowledge Graph	สกัด entity/relationship จากเอกสาร เพื่อตอบคำถามเชิงความสัมพันธ์
Retriever	กลไกค้นคืนชิ้นข้อมูลที่เกี่ยวข้องที่สุดกับคำถาม
LLM (Generator)	นำข้อมูลที่ค้นได้มาสังเคราะห์เป็นคำตอบ
Prompt/Instruction Layer	ควบคุมพฤติกรรมการตอบ (ภาษา, ขอบเขต, รูปแบบการอ้างอิง)



เครื่องมือสำเร็จรูป

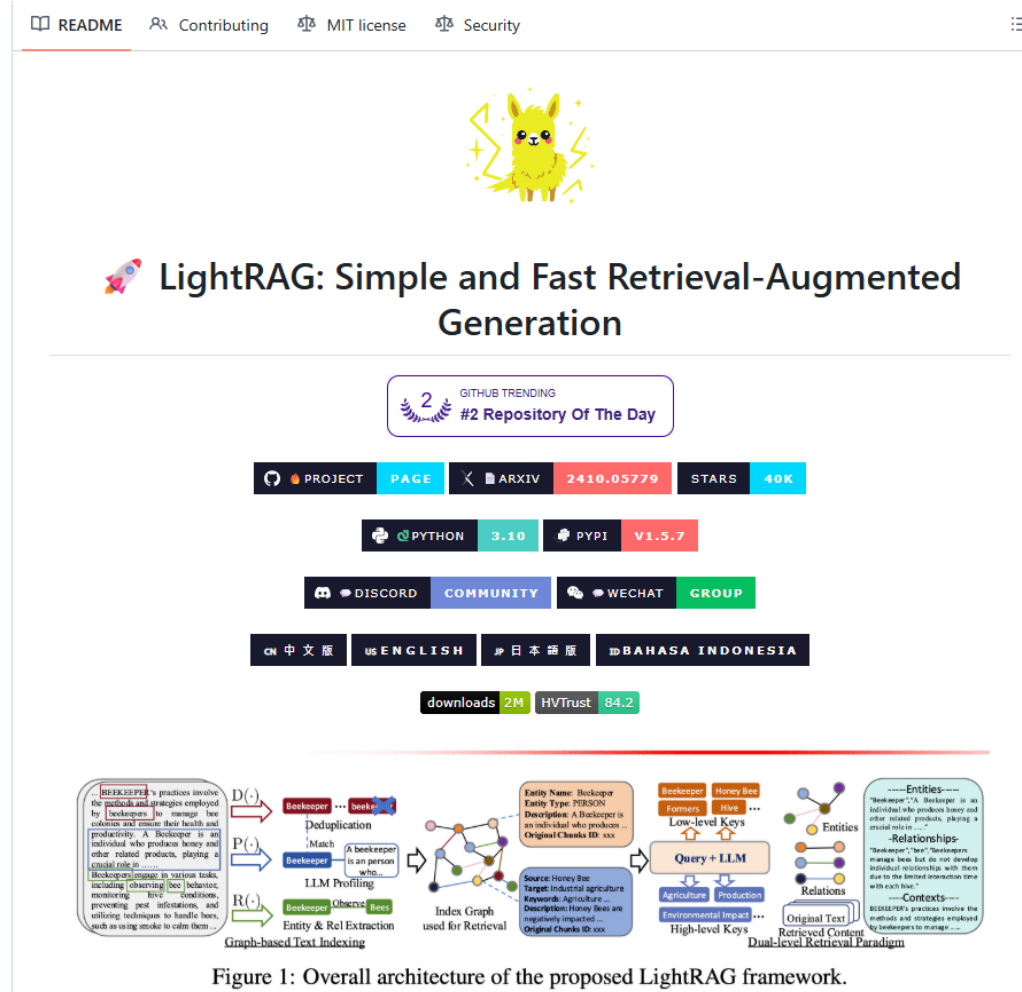
กลุ่มเครื่องมือในตลาดปัจจุบัน

กลุ่มเครื่องมือ	ตัวอย่าง	จุดเด่น
Framework/Library สำหรับนักพัฒนา	LangChain, LlamaIndex, Haystack	ยืดหยุ่นสูง แต่ต้องเขียนโค้ดประกอบเอง
RAG-as-a-Service / All-in-one server	LightRAG (ใช้ใน workshop)	ครบวงจรในตัวเดียว ติดตั้งง่ายผ่าน Docker
No-code/Low-code platform	Dify, Flowise	สร้าง pipeline ผ่าน UI ลากวาง เหมาะสำหรับผู้ไม่ถนัดโค้ด
Managed/Cloud platform	OpenAI, Azure AI Search, Vertex AI Search	สะดวก แต่ผูกกับผู้ให้บริการและมีค่าใช้จ่ายตามการใช้งาน



เกณฑ์การเลือกเครื่องมือ

- 01 **ขนาดคลังข้อมูล** — ปริมาณเอกสารและอัตราการเติบโตของคลังความรู้
- 02 **งบประมาณ** — ค่าใช้จ่าย host เอง vs. ใช้บริการ cloud/managed
- 03 **ต้องการ Knowledge Graph หรือไม่** — งานที่ต้องตอบเชิงความสัมพันธ์ ควรเลือกเครื่องมือที่รองรับ
- 04 **ทีมมีนักพัฒนาหรือไม่** — มีนักพัฒนา → framework ยืดหยุ่น / ไม่มี → no-code platform
- 05 **ต้องการ Host เองหรือใช้ Cloud** — ควบคุมข้อมูลเองได้เต็มที่ vs. ความสะดวกของบริการสำเร็จรูป





Workshop: ภาพรวมขั้นตอนลงมือทำ

ต่อยอดจาก prepare-lesson.md · ขั้นตอนที่ 1-4

1

ดาวน์โหลดและติดตั้ง LightRAG

บนเครื่อง local

2

ตั้งค่า LLM/Embedding ผ่าน OpenRouter

ไฟล์ .env

3

รัน Docker Compose และนำเข้าเอกสารเฉพาะทาง

เช่น คู่มือ/ระเบียบของหน่วยงาน เพื่อสร้างคลังความรู้

4

ทดสอบถาม-ตอบ

ผ่าน LightRAG WebUI และ Postman



ตัวอย่าง: บังคับภาษาไทยและจำกัดขอบเขตคลังข้อมูล

POST /query

```
// บังคับตอบภาษาไทย และจำกัดคำตอบให้อยู่ในขอบเขตคลังข้อมูล
{
  "query": "คำถามทดสอบ",
  "mode": "hybrid",
  "user_prompt": "ตอบคำถามจากคลัง data และสนทนาเป็นภาษาไทยเท่านั้น  
ถ้าไม่พบคำตอบให้ตอบกลับว่า 'ไม่สามารถให้คำตอบได้เนื่องจากอยู่นอกขอบเขตคลังข้อมูล'"
}
```



POST /query — query และ mode

query (string, บังคับ) คือคำถามที่จะถาม

mode	ทำงานอย่างไร
local	เน้น entity เฉพาะจุดและความสัมพันธ์โดยตรงของมัน
global	วิเคราะห์รูปแบบ/ความสัมพันธ์ในภาพกว้างทั่ว knowledge graph
hybrid	ผสาน local + global
naive	vector similarity search ธรรมดา ไม่ใช้ knowledge graph
mix	ผสาน knowledge graph + vector retrieval (default, แนะนำ)
bypass	ยังตรงเข้า LLM ไม่ค้นคลังข้อมูลใด ๆ (debug/เทียบผล)



ควบคุมปริมาณ/ขอบเขตข้อมูลที่ดึงมา

- 01 top_k** — จำนวนรายการบนสุดที่ดึงมา (entity ใน local, relationship ใน global) ค่า default จาก env TOP_K
- 02 chunk_top_k** — จำนวน text chunk ที่ดึงจาก vector search แล้วคัดเหลือหลัง rerank ค่า default จาก env CHUNK_TOP_K
- 03 max_entity / max_relation / max_total_tokens** — งบ token สูงสุดของแต่ละส่วน คุณไม่ให้ context ยาวเกินจนแพงหรือเกิน context window
- 04 hl_keywords / ll_keywords** — keyword ที่อยากเน้นค้น ถ้าไม่ใส่ระบบให้ LLM สร้างเอง (ใส่เพื่อข้าม LLM call ขึ้นสกัด keyword)



ควบคุมรูปแบบ/พฤติกรรมคำตอบ

- 01 response_type** — รูปแบบคำตอบ เช่น Multiple Paragraphs, Single Paragraph, Bullet Points
- 02 user_prompt** — คำสั่งเพิ่มเติมที่แทรกใน "Additional Instructions" ไม่กระทบการค้นคืน แต่กำหนดวิธีตอบ เช่น บังคับภาษาไทย
- 03 disable_user_prompt_prefix** — ปิด global prefix ฝั่ง server ให้ client คุมข้อความ instruction เองทั้งหมด (default false)
- 04 conversation_history** — ประวัติบทสนทนา ส่งให้ LLM ใช้เป็นบริบทตอบเท่านั้น ไม่ถูกใช้ค้นคืนข้อมูล
- 05 enable_rerank** — เปิด/ปิดการ rerank chunk ที่ดึงมา (default true)



ควบคุมสิ่งที่ตอบกลับมา

- 01 **only_need_context** — คัดแค่ context ที่ค้นได้ ไม่เรียก LLM สร้างคำตอบ (debug การค้นคืน)
- 02 **only_need_prompt** — คัดแค่ prompt ที่ประกอบขึ้น โดยไม่ยิงไป LLM (ตรวจสอบ prompt สุดท้าย)
- 03 **include_references** — แบนรายการอ้างอิง (แหล่งที่มา) มาด้วยหรือไม่ (default true)
- 04 **include_chunk_content** — ใส่เนื้อหาข้อความจริงของ chunk ไว้ใน reference ด้วย (ใช้ evaluate/debug)
- 05 **include_progress** — ส่ง event ความคืบหน้า เฉพาะ /query/stream เท่านั้น
- 06 **stream** — เปิด streaming หรือไม่ (default false สำหรับ /query, true สำหรับ /query/stream)